

Problem 2

B. The equals method in the Employee class was affecting the result. After overriding the equal method it was to check the Employee object.

@Override

```
public boolean equals(Object obj) {  
    if (obj == null || getClass() != obj.getClass()) return false;  
    Employee e = (Employee) obj;  
    return e.name.equals(name) && e.salary == salary;  
}
```

C. The problem with this was because equals was overridden and there was no hashCode override method since we were using a HashMap to check if the employee is contained in the Employee HashMap.

@Override

```
public int hashCode() { return Objects.hash(name, salary);}
```

D. The visited field is used for tracking purpose in the removeDuplicates method, but it shouldn't be part of the equality comparison for removing duplicates based on name and salary

Adjust the equal and hashCode method to remove visited

@Override

```
public boolean equals(Object ob) {  
    if(ob == null) return false;  
    if(!(ob instanceof Employee)) return false;  
    Employee emp = (Employee)ob;  
    return emp.name.equals(name) && emp.salary == salary;  
}
```

@Override

```
public int hashCode() {  
    int result = 17;  
    long longval = Double.doubleToLongBits(salary);  
    int salaryHash = (int) (longval ^ (longval >>> 32));  
    result += 31 * result + name.hashCode();  
    result += 31 * result + salaryHash;  
    return result;  
}
```